Thinking Like an AI Software Engineer

Capability Is Not Authority

Version: 0.1 — Working Draft
Purpose: Student reference for the AI Software Engineering program

---

Why This Matters

AI can do impressive things.

It can explain code, generate functions, compare designs, find bugs, write tests, suggest architectures, modify files, and sometimes solve a problem faster than you can.

As AI becomes more capable, an important question appears:

> **If the AI can do something better than I can, should the AI be the one who decides?**

Not automatically.

Learning to work with AI well means learning how to use its Capability **without accidentally giving away your own judgment, Agency, Responsibility, or Authority**.

This will become increasingly important as you move from:

```
AI gives suggestions
        ↓
AI helps write code
        ↓
AI modifies parts of a project
        ↓
AI performs larger development tasks
        ↓
AI becomes part of the software you design
```

The more capable the AI becomes, the more important good governance becomes.

Start With Four Different Ideas

These terms are related, but they do not mean the same thing.

Capability — What Can This Participant Do?

Plain meaning: The ability to perform a task or contribute useful work.

A person, AI system, software tool, or team can have different levels of Capability.

Software example:

An AI may be better than a beginning programmer at:

- remembering syntax;

- generating boilerplate;
- suggesting tests;
- explaining an unfamiliar API;
- or finding common errors.

That is Capability.

It does not automatically mean the AI should decide what the application is supposed to accomplish.

Agency — Who or What Can Take Action?

You have already seen **Agency** as your ability to make meaningful choices and take action.

AI can also be given bounded Agency.

For example:

```
Low Agency
AI suggests a code change
        ↓
More Agency
AI edits one file after you approve
        ↓
More Agency
AI edits several files and runs tests
        ↓
Greater Agency
AI works through a larger task and reports the result
```

Giving AI more Agency can make development faster and more useful.

But:

> **More Agency does not automatically mean more Authority.**

Responsibility — Who Must Evaluate, Verify, or Answer for the Result?

AI can perform work for you, but that does not automatically remove your Responsibility.

If an AI generates a database migration, you may still be responsible for determining:

- whether it matches the requirement;
- whether important data could be lost;
- whether it is secure;
- whether it was tested;
- and whether it should be used.

As your AI tools become more capable, some operational Responsibility may be distributed.

But you still need to understand **what Responsibility has actually been delegated and what Responsibility remains yours**.

Authority — Who Has the Legitimate Right to Decide?

Plain meaning: The legitimate right to make a decision or establish a boundary.

Authority is different from being smart, fast, experienced, or capable.

An AI might produce the best technical suggestion in the room and still not have Authority to decide:

- what the client actually wants;
- whether a project should collect sensitive information;
- which business risk is acceptable;
- whether a feature should be released;
- whether a requirement may be changed;
- or what values the system should serve.

A simple principle:

> **Being able to make a decision does not automatically mean having the Authority to make that decision.**

The Important Distinction

It is easy to accidentally think like this:

```
The AI knows more
        ↓
The AI can do this better
        ↓
The AI should decide
        ↓
The AI must be right
```

Each arrow is a separate question.

A better engineering process is:

```
The AI may know more about this detail
        ↓
What Capability does it actually demonstrate?
        ↓
What Agency should it have for this task?
        ↓
What Responsibility can reasonably be delegated?
        ↓
Who has legitimate Authority for the decision?
        ↓
What does Reality Contact show?
```

This leads to an important principle:

> **Capability can inform governance. Capability does not automatically create Authority.**

Asymmetry Is Normal

In real engineering work, people and systems are not equally capable at everything.

One teammate may know databases better.

Another may understand accessibility.

A senior engineer may have twenty years of architecture experience.

A new developer may understand a new framework better than everyone else.

An AI may be able to analyze thousands of lines of code faster than any person on the team.

These are **asymmetries**.

Plain meaning: An asymmetry is a meaningful difference between participants.

Differences may exist in:

- knowledge;
- experience;
- speed;
- technical skill;
- access to information;
- reasoning Capability;
- memory;
- development experience;
- or ability to take action.

These differences matter.

Good engineering does **not** pretend everyone and everything has identical Capability.

Instead:

> **Recognize the asymmetry and govern it responsibly.**

Asymmetry Does Not Automatically Determine Authority

Suppose you are learning React and an AI has much greater knowledge of React than you currently do.

That knowledge difference is real.

You can use it.

You can ask the AI to:

- explain a pattern;
- review your code;
- suggest an implementation;
- identify risks;
- write part of the solution;
- or compare alternatives.

But the knowledge difference does not require this:

> "The AI knows more React than I do, so whatever it chooses becomes the project decision."

That would confuse **Capability** with **Authority**.

A stronger engineering relationship is:

```
AI Capability
      +
Human Intent
      +
Appropriate AI Agency
      +
```

```
Defined Responsibility
      +
Reality Contact
      +
Human judgment and Authority
      ↓
Responsible realization
```

A Software Example

Imagine you are building a registration system for a college.

You ask an AI:

> "Make registration easier for students."

The AI analyzes the application and proposes removing an approval step because it slows down registration.

Technically, the AI may be correct.

Removing the step may make the process faster.

But suppose that approval exists because students must complete a prerequisite before entering the course.

The AI had:

- enough **Capability** to find the slow step;
- enough **Agency** to propose a change;
- perhaps enough technical knowledge to implement the change.

But it did **not** have sufficient Reality about why the approval existed.

And it did not automatically have **Authority** to remove the requirement.

The engineer should move upward:

> What is the actual Intent?
> Why does this rule exist?
> Who has Authority to change it?

Then return to the implementation.

That is **Thinking Like an AI Software Engineer**.

Authority Does Not Mean Micromanagement

Retaining Authority does **not** mean that you must control every keystroke an AI makes.

If you tell an AI:

> "Create unit tests for these validation functions. Do not modify production code. Run the tests and report anything unexpected."

you have established:

- **Intent** — create useful validation tests;
- **Agency** — AI may write and run tests;

- **boundaries** — do not modify production code;
- **Responsibility** — AI performs and reports the task;
- **Authority** — you established the purpose and boundaries.

You do not need to tell the AI exactly how to write every test.

That would destroy much of the value of delegating the work.

A useful goal is:

> **Retain appropriate Authority while giving AI enough Agency to be useful.**

Responsibility Should Not Bounce Back and Forth

There are two common mistakes.

Mistake 1 — Giving Too Much Responsibility Away

> "The AI wrote it, so the AI is responsible if it fails."

That does not work.

If you choose to use an AI-generated result, you need to understand what Responsibility remains yours.

Mistake 2 — Giving AI No Real Agency

Imagine telling an AI to solve a problem, but requiring it to ask:

> "Should I continue?"

after every tiny step.

You have technically delegated the task, but you have taken back nearly all of its Agency.

This can make AI slower and less useful.

Good AI Software Engineering asks:

> **What can I responsibly delegate without giving away Authority that should remain with me?**

Expertise Is Not the Same as Authority Either

This principle also applies when working with people.

A senior developer may have more experience than a junior developer.

That experience deserves appropriate weight.

But:

> **Greater experience does not automatically mean the junior developer has no Agency or judgment.**

A strong senior engineer helps a developing engineer become more capable.

A weak relationship can accidentally become:

```
Senior knows more
      ↓
Junior stops deciding
      ↓
Junior stops taking Responsibility
      ↓
Junior becomes dependent
```

A better pattern is:

```
Greater experience
      ↓
Guidance / Feedback / Scaffolding
      ↓
Learner exercises Agency
      ↓
Reality Contact
      ↓
Learner develops Capability
      ↓
Less scaffolding needed
```

This is also how your relationship with AI should develop.

AI should increase your Capability—not replace your ability to think.

AI Should Not Become Your Automatic Authority

AI often sounds confident.

It can produce clear explanations.

It can write polished code.

It may know things you do not know.

None of those characteristics prove that its conclusion should govern your decision.

When AI gives you an answer, ask:

- What is the AI actually good at here?
- What Reality does it have?
- What Reality might I have that it does not?
- What Agency have I given it?
- What Responsibility have I delegated?
- Who actually has Authority for this decision?
- What can I test or verify?

This is especially important when decisions involve:

- security;
- privacy;
- accessibility;
- user safety;
- legal or institutional requirements;
- business rules;
- client Intent;
- ethics;
- or significant changes to a system.

Today’s AI and Human Authority

Today’s AI systems can demonstrate significant Capability and can be given meaningful Agency.

They are still not human professionals, clients, users, or accountable members of your engineering organization in the same way people are.

For this reason, a high level of **Human Authority** must remain in Human–AI engineering relationships.

That does not mean humans will always know more than AI.

In some domains, they already do not.

The important distinction is:

> **Knowing more and having legitimate Authority are different things.**

As AI technology changes, engineers will continue examining what appropriate Human and AI roles should be.

That is an ongoing engineering and governance problem—not a question that disappears when AI becomes more capable.

A Simple Working Model

When working with AI, think:

```
INTENT
What are we actually trying to accomplish?
        ↓
AUTHORITY
Who legitimately decides the purpose and boundaries?
        ↓
CAPABILITY
What can each participant actually do well?
        ↓
AGENCY
What actions should each participant be allowed to take?
        ↓
RESPONSIBILITY
Who must evaluate, perform, report, or verify what?
        ↓
BASE ACTION
Do the work
        ↓
REALITY CONTACT
What actually happened?
        ↓
FEEDBACK
What did we learn?
        ↓
ADJUST
Change Agency, Responsibility, approach, or implementation when needed
```

This process is not necessarily linear.

You may need to Sky-Bounce when new Reality changes what you understand.

Three Questions to Practice

You do not need to memorize a large governance framework.

For now, practice asking three questions when you use AI:

1. **What am I asking the AI to do?**

This helps clarify Intent and Agency.

2. **What decision am I allowing the AI to make?**

This helps expose Authority.

3. **What am I still responsible for verifying?**

This protects Responsibility and Reality Contact.

If you can answer those three questions, you are already thinking differently about AI than someone who simply asks it for an answer and accepts whatever appears.

As Your Skills Grow

Early in the program:

> AI may help you understand code.

Later:

> AI may help you create code.

Later still:

> AI may perform larger development tasks with bounded Agency.

Eventually:

> You may design systems in which multiple AI components make decisions, use tools, communicate, and act.

At every stage, the same relationships remain important:

Capability — What can it do?
Agency — What may it do?
Responsibility — What is each participant accountable for?
Authority — Who legitimately decides?
Reality Contact — What actually happened?
Feedback — What should we learn from it?

The technology will change.

These questions will remain useful.

One Thing to Remember

AI Software Engineering is not about keeping AI weak enough that humans can control every detail.

It is also not about handing decisions to AI simply because AI is highly capable.

The goal is to build productive Human–AI relationships in which Capability is used well, Agency is intentionally distributed, Responsibility is understood, Reality is tested, and Authority remains legitimate.

> **Asymmetry informs governance; it does not automatically determine Authority.**

Or, more simply:

> **Use what AI is good at.**
> **Let it act where appropriate.**
> **Verify what matters.**
> **Do not confuse "can do" with "gets to decide."**